SD	Sistemas Distribuidos
26/27	Water Management
	Tutorial: Introducción a AWS-Labs en configuración CI/CD

Preámbulo

Objetivo: Familiarizarse con el flujo de despliegue automatizado (CI/CD) utilizando AWS Labs antes de aplicarlo en las prácticas de la asignatura de Sistemas Distribuidos.

En plataformas como Railway, el proceso de despliegue está completamente automatizado: vinculas tu repositorio, haces clic en "Añadir servicio" para tu base de datos o broker, y la plataforma gestiona los servidores, redes y actualizaciones de forma invisible.

En AWS, replicaremos exactamente esa misma arquitectura (un entorno Web y un entorno de Datos), pero asumiendo el control de la infraestructura. Construiremos nuestra propia "tubería" de integración continua y configuraremos los servidores manualmente.

Prerrequisitos

Antes de tocar la consola de AWS, asegúrate de cumplir con los siguientes requisitos en tu ordenador:

1. **Cuentas activas:**
    - Cuenta de AWS (Laboratorios de la universidad o cuenta personal).
    - Cuenta de GitHub.
    - Cuenta de Docker Hub (gratuita).
2. **Herramientas locales:**
    - Visual Studio Code (o tu editor de código preferido).
    - Git instalado en tu ordenador. Para evitar errores al subir código, abre una terminal y asegúrate de haber configurado tu identidad ejecutando:

```
Bash

git config --global user.name "Tu Nombre"

git config --global user.email "tu_correo@ejemplo.com"
```

3. **Archivos del proyecto listos:**

    - Debes tener una carpeta en tu ordenador con el código de tu aplicación web (por ejemplo, un archivo server.py configurado para escuchar en el puerto 8080).

    - Un archivo llamado Dockerfile (sin extensión) en la misma carpeta, que contenga las instrucciones para empaquetar tu código y que incluya la línea EXPOSE 8080.

    - Haber ejecutado git init, hecho el primer commit y subido este código inicial a un repositorio en GitHub.

Paso 1: El Backend de Datos (El equivalente a "Añadir Servicio" en Railway)

En Railway, al añadir Kafka o PostgreSQL, la plataforma levanta un contenedor en una red privada aislada. En AWS, alquilaremos una máquina virtual pura (Amazon EC2) para tener control total sobre el almacenamiento y los puertos del motor de datos.

> *Nota sobre arquitectura y seguridad: Para esta práctica de laboratorio, AWS colocará nuestra máquina en una subred pública (asignándole una IP pública) para que podáis ejecutar scripts locales y conectaros desde la universidad. En un entorno profesional, bases de datos y motores como Kafka siempre se aíslan en una subred privada sin salida a internet, permitiendo solo conexiones internas.*

1.1 Levantar el servidor

- En la consola de AWS, ve a EC2 > Lanzar instancia.
- Nombre: Broker-Kafka.
- Imágenes (AMI): Selecciona Ubuntu (necesario para usar comandos de Linux estándar).
- Tipo de instancia: t2.micro (apto para la capa gratuita).
- Par de claves: Selecciona "Proceder sin un par de claves".
- Red (Cortafuegos): Marca "Seleccionar un grupo de seguridad existente" y elige el llamado default. Pulsa Lanzar.

1.2 Configurar el cortafuegos (Security Group)

- En el menú izquierdo de EC2, ve a Grupos de seguridad.
- Selecciona el grupo default y pulsa Editar reglas de entrada.

- Añade: Tipo SSH, Puerto 22, Origen 0.0.0.0/0 (permite abrir la terminal desde el navegador).

- Añade: Tipo TCP personalizado, Puerto 9092, Origen 0.0.0.0/0 (permite recibir mensajes en Kafka).

- Guarda las reglas.

1.3 Instalar y encender Kafka

- Ve a Instancias, selecciona Broker-Kafka, copia su Dirección IPv4 pública y pulsa Conectar (pestaña EC2 Instance Connect).

- En la terminal del navegador, instala Docker:

```
Bash

sudo apt update -y

sudo apt install docker.io -y
```

- Ejecuta el contenedor de Kafka sustituyendo TU_IP_PUBLICA por la tuya. Usamos la versión 3.7.0 de bitnamilegacy por estabilidad:

```
Bash

sudo docker run -d --name kafka -p 9092:9092 \
 -e KAFKA_CFG_NODE_ID=0 \
 -e KAFKA_CFG_PROCESS_ROLES=controller,broker \
 -e KAFKA_CFG_LISTENERS=PLAINTEXT://:9092,CONTROLLER://:9093 \
 -e
KAFKA_CFG_LISTENER_SECURITY_PROTOCOL_MAP=CONTROLLER:PLAINTEXT,PLAIN
TEXT:PLAINTEXT \
 -e KAFKA_CFG_CONTROLLER_QUORUM_VOTERS=0@localhost:9093 \
 -e KAFKA_CFG_CONTROLLER_LISTENER_NAMES=CONTROLLER \
 -e KAFKA_CFG_ADVERTISED_LISTENERS=PLAINTEXT://TU_IP_PUBLICA:9092 \
 bitnamilegacy/kafka:3.7.0
```

Paso 2: Conectar el Repositorio (El equivalente a "Vincular GitHub")

Railway lee tu código y construye el servidor automáticamente. Para replicar esto en AWS, usaremos GitHub Actions como nuestra fábrica y Docker Hub como el almacén público donde guardaremos la aplicación web empaquetada.

2.1 Preparar Docker Hub

- Entra en hub.docker.com con tu cuenta.
- Haz clic en Create Repository.
- Nombre: practica-aws.
- Visibilidad: Marca Public para que AWS pueda leerlo sin restricciones.

2.2 Configurar credenciales en GitHub

- En tu repositorio de GitHub, ve a Settings > Secrets and variables > Actions.
- Haz clic en New repository secret.
- Crea un secreto llamado DOCKER_USERNAME con tu usuario de Docker Hub.
- Crea otro secreto llamado DOCKER_PASSWORD con tu contraseña.

2.3 Crear el archivo de automatización

- En tu ordenador (VS Code), en la raíz de tu proyecto, crea la ruta de carpetas .github/workflows/.
- Dentro, crea el archivo deploy.yml y pega este código:

```
YAML
name: Build and Push to Docker Hub
on:
  push:
    branches:
      - main # Asegúrate de que coincide con el nombre de tu rama principal
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
```

```
      - name: Checkout del código
        uses: actions/checkout@v3
      - name: Iniciar sesión en Docker Hub
        uses: docker/login-action@v2
        with:
          username: ${{ secrets.DOCKER_USERNAME }}
          password: ${{ secrets.DOCKER_PASSWORD }}
      - name: Construir y subir imagen
        uses: docker/build-push-action@v4
        with:
          context: .
          push: true
          tags: ${{ secrets.DOCKER_USERNAME }}/practica-aws:latest
```

- Sube los cambios con git add ., git commit -m "Pipeline" y git push. Al terminar, verás un tic verde en la pestaña "Actions" de GitHub.

Paso 3: El Frontend Web (El equivalente a la "URL generada" de Railway)

Con la imagen de nuestra web ya fabricada y guardada en Docker Hub, usaremos Amazon ECS Express para que AWS proporcione los servidores físicos, el balanceador de carga y la URL pública, manteniéndolo aislado de nuestra base de datos.

- En la consola de AWS, busca ECS y entra en Modo exprés.
- URI de imagen: Escribe la ruta a tu almacén: docker.io/TU_USUARIO/practica-aws:latest.
- Roles: Deja las opciones predeterminadas para que AWS genere los permisos.
- Puerto del contenedor: Borra el puerto 80 por defecto y escribe 8080 (o el puerto que hayas expuesto en tu Dockerfile).
- Pulsa Implementar. AWS tardará unos minutos en configurar el entorno y te proporcionará un enlace público funcional.

Paso 4: Cómo actualizar el código (Iteración del ciclo de vida)

En Railway, cada git push actualiza la web automáticamente en producción. En nuestra arquitectura de AWS, tenemos la automatización dividida en dos fases por seguridad:

La parte automática: Al hacer un git push en VS Code, GitHub Actions empaquetará el nuevo código y actualizará el contenedor en Docker Hub de forma autónoma.

La parte manual (Despliegue): Para decirle a la web que aplique los cambios, ve a la consola de AWS ECS, selecciona tu servicio web, pulsa Actualizar, marca la casilla Forzar nueva implementación y guarda los cambios. AWS descargará silenciosamente la nueva versión de Docker Hub sin interrumpir el servicio a los usuarios.